

1) Crie uma classe chamada CalculadoraController no seu projeto Spring Boot (pode ser no pacote controller ou direto na raiz do pacote principal) e implemente as quatro operações básicas usando @RequestParam para receber os valores.

Requisitos obrigatórios:

- A classe deve ter a anotação @RestController
- O mapping da classe deve ser /calculadora
- Cada operação deve ter seu próprio @GetMapping com um path descritivo
- Os parâmetros a e b devem usar @RequestParam
- A operação de dividir deve tratar o caso de divisão por zero retornando uma mensagem de erro em vez de explodir a aplicação

```
@RestController
@RequestMapping("/calculadora")
public class CalculadoraController {

    // 1. Somar - GET /calculadora/somar?a=10&b=5
    @GetMapping("/somar")
    public double somar(@RequestParam double a, @RequestParam double b) {
        // Escreva aqui
    }

    // 2. Subtrair - GET /calculadora/subtrair?a=10&b=5
    // ??? implemente aqui

    // 3. Multiplicar - GET /calculadora/multiplicar?a=4&b=3
    // ??? implemente aqui

    // 4. Dividir - GET /calculadora/dividir?a=10&b=2
    //   Atenção: trate a divisão por zero!
    // ??? implemente aqui
}
```

b) Teste todos os endpoints no Postman e preencha a tabela de resultados:

URL testada no Postman	Status HTTP	Resultado retornado
GET /calculadora/somar?a=20&b=8	200 ok	28
GET /calculadora/subtrair?a=20&b=8	200 ok	12.0
GET /calculadora/multiplicar?a=7&b=6	200 ok	42
GET /calculadora/dividir?a=15&b=3	200 ok	5.0
GET /calculadora/dividir?a=10&b=0	400 bad request	Valor inválido!

c)

Por que usamos @GetMapping em vez de @PostMapping para os endpoints da calculadora? Faz sentido usar GET para operações matemáticas? Justifique baseando-se nos conceitos de REST da Aula 01.

Conclusão:

Sim, porque o método precisa retornar um valor, não criar um novo objeto.

2) Implemente uma API completa para gerenciar veículos seguindo os passos abaixo:

Passo 1 : Crie a classe Veículo no pacote domain com os atributos abaixo:

Atributo	Tipo	Observação
id	Long	Identificador único
marca	String	Ex: Toyota, Honda, Fiat
modelo	String	Ex: Corolla, Civic, Uno

⚠ Lembre-se de criar AMBOS os construtores: com argumentos E o construtor vazio (necessário para o Jackson funcionar no @RequestBody).

Passo 2 : Crie o VeiculoController no pacote controller e implemente os endpoints abaixo:

Verbo	URI	O que deve fazer
GET	/veiculos	Retorna a lista completa de veículos em JSON
GET	/veiculos/{id}	Retorna o veículo com o id informado (usa @PathVariable)
POST	/veiculos	Recebe um veículo no body e adiciona na lista. Retornar status 201.
PUT	/veiculos/{id}	Atualiza o veículo com o id informado substituindo marca e modelo
DELETE	/veiculos/{id}	Remove o veículo com o id informado da lista

Dica: inicialize a lista com pelo menos 3 veículos no bloco static {}

Passo 3 : Teste todos os endpoints no Postman e preencha:

Endpoint testado	Status retornado	Resultado / observação
GET /veiculos	200 ok	lista de veículos
GET /veiculos/2	200 ok	veiculo id 12347
POST /veiculos (body com novo veículo)	201 created	veiculo criado
PUT /veiculos/1 (body com atualização)	200 ok	veiculo alterado
DELETE /veiculos/3	200 ok	veiculo deletado

3) Adicione ao seu VeiculoController um novo endpoint que permita filtrar veículos por marca usando @RequestParam:

Verbo + URI	GET /veiculos/buscar?marca=Toyota
Comportamento	Deve retornar APENAS os veículos cuja marca seja igual ao valor informado. Se nenhum for encontrado, retorne uma lista vazia.
Anotação	@RequestParam String marca (parâmetro na query string)

4) Escolha UMA das entidades abaixo e implemente o CRUD completo da mesma forma que fizemos com Aluno e Veículo:

Opção	Atributos sugeridos	Endpoint base
 Produto	id (Long), nome (String), preco (Double)	/produtos
 Livro	id (Long), titulo (String), autor (String)	/livros
 Funcionário	id (Long), nome (String), cargo (String)	/funcionarios